SUMMARY: PANDAS-III

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand how to handle missing values
- Understand the other DataFrame operations such as statistical operations, frequency analysis, data aggregations, sorting a DataFrame, idxmax function, GROUP BY functions, Altering the index.
- Understand how to export a DataFrame to a CSV or Excel file.

KEY POINTS AND EXAMPLES:

Understand the other DataFrame operations

Statistical Operation

Statistical operations are fundamental to data analysis as they provide insights into the data's underlying patterns and characteristics.
In pandas, a powerful data manipulation library in Python, these operations enable you to summarize, describe your data efficiently.

A. **Mean:** The mean (average) of the values of each column of a DataFrame. It is only applicable for numeric values.

```
Purchase.mean()
```

The output will be:

```
id                            3000.5
purch_date       2019-05-13 20:18:00
customer_num              500.889333
product_num                30.140667
amount                     10.576167
paid                      548.871353
dtype: object
```

The mean() function is giving a nonsensical result for the purch_date column because it tries to interpret the datetime values as numeric values.

```
# Select only numeric columns
numeric_columns = Purchase.select_dtypes(include=['number'])
```

```
# Compute mean for numeric columns
mean_values = numeric_columns.mean()

# Print the mean values
print("\nMean values for numeric columns:")
print(mean_values)
```

Purchase.select_dtypes(include=['number']) filters out only the numeric columns.

The output will be:

```
Mean values for numeric columns:
id              3000.500000
customer_num     500.889333
product_num       30.140667
amount            10.576167
paid             548.871353
dtype: float64
```

you can perform the statistical operations on specific columns in a pandas DataFrame.

```
result = Purchase['paid'].mean()
print(result)
```

result = Purchase['paid'].mean()
print(result)

The output will be:

```
548.8713533333333
```

Note: If the DataFrame contains non-numeric columns (such as strings), applying mean() directly will result in an error.

```
Customers.mean()
```

The output will be:

```
TypeError: Could not convert.....
```

```
Products.mean()
```

The output will be:

```
TypeError: Could not convert....
```

To handle this and correctly calculate the mean for numeric columns you can filter the DataFrame to include only columns with numeric data types.

B. **Median:** It displays the middle value of the data. This function will display the median of the values of each column of a DataFrame. It is only applicable for numeric values.

```
Purchase.median()
```

The output will be:

```
id                            3000.5
purch_date       2019-04-26 00:00:00
customer_num                  511.0
product_num                    30.0
amount                         11.0
paid                          422.7
dtype: object
```

The median calculation for non-numeric column like purch_date might not make sense.

```
# Select only numeric columns
numeric_columns = Purchase.select_dtypes(include=['number'])

# Compute median for numeric columns
median_values = numeric_columns.median()

# Print the median values
print("Median values for numeric columns:")
print(median_values)
```

The output will be:

```
Median values for numeric columns:
id              3000.5
customer_num     511.0
product_num       30.0
amount            11.0
paid             422.7
dtype: float64
```

C. **Mode:** The mode is defined as the value that appears the most number of times in a data. This function will display the mode of each column or row of the DataFrame. The mode can be useful for categorical or discrete variables to understand the most common values in the dataset.

```
Customers.mode()
```

The output will be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
0	1	Berty	Sedworth	Unknown	Male	4.0	Anniversary	Place	Washington	Texas	60636.0
1	2	NaN	NaN	NaN	NaN	NaN	Pankratz	NaN	NaN	NaN	NaN
2	3	NaN	NaN	NaN	NaN	NaN	Washington	NaN	NaN	NaN	NaN
3	4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
...	...	...	...	...	...	...	...	...	...	...	...
995	996	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
996	997	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
997	998	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
998	999	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
999	1000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

1000 rows × 11 columns

The output displays the mode values row-wise, where each row represents the mode for one column.

In cases where there is no unique mode (multiple values with the same frequency), additional rows with NaN are displayed to indicate ambiguity.

D. **Calculating Maximum Values:** DataFrame.max() is used to calculate the maximum values from the DataFrame.

```
# Calculate the maximum for numerical columns in the Purchase
DataFrame
max_values = Purchase.max(numeric_only=True)
print("Maximum values:\n", max_values)
```

Purchase.max(numeric_only=True): This calculates the maximum values for numerical columns in the Purchase DataFrame.

The output will be:

```
Maximum values:
 id              6000.0
customer_num    1000.0
product_num       60.0
amount            20.0
paid            1990.8
dtype: float64
```

When performing operations like finding the maximum and minimum values using max() and min() functions, pandas often returns results in the float64 data type by default, even if the original data type is integer.

However, you can convert the result back to integers if you need to maintain the original integer type.

```
# Convert the results to integers where appropriate
max_values = max_values.astype(int)
print("Maximum values:\n", max_values)
```

The output will be:

```
Maximum values:
 id              6000
customer_num    1000
product_num       60
amount            20
paid            1990
dtype: int32
```

E. **Calculating Minimum Values:** DataFrame.min() is used to display the minimum values from the DataFrame.

```
# Calculate the minimum for numerical columns in the Purchase
DataFrame
min_values = Purchase.min(numeric_only=True)
# Convert the results to integers where appropriate
min_values = min_values.astype(int)
```

```
print("Minimum values:\n", min_values)
```

The output will be:

```
Minimum values:
 id              1
customer_num    1
product_num     1
amount          1
paid            3
dtype: int32
```

F. **Calculating Sum of Values:** DataFrame.sum() will display the sum of the values from the DataFrame.

```
# Calculate the sum for numerical columns in the Purchase
DataFrame
sum_values = Purchase.sum(numeric_only=True)

# Print the result
print("Sum of values:\n", sum_values)
```

The output will be:

```
Sum of values:
 id              18003000.00
customer_num     3005336.00
product_num       180844.00
amount             63457.00
paid             3293228.12
dtype: float64
```

Note: The sum of the id, customer_num, and product_num columns appears unusually high because id values are typically unique identifiers, and summing them might not provide meaningful insight. Summing the customer_num and product_num columns might be more relevant in certain contexts, but generally, summing identifiers or counts may not offer significant analytical value.

G. **Calculating Number of Values:** DataFrame.count() will display the total number of values

```
# Count non-null values in each column
column_counts = Products.count()
```

```
print("Count of non-null values in each column:")
print(column_counts)
```

The output will be:

```
Count of non-null values in each column:
id         60
product    60
cost       60
company    60
dtype: int64
```

Frequency Analysis

Use .value_counts() to compute the frequency distribution of categorical data, revealing the count of unique values within a column. This method is particularly useful for understanding the distribution and occurrence of different categories in dataset.

Let's say we want to find out how often each company appears in the company column of the Products DataFrame.

```
# Calculate the frequency distribution of the 'company' column in
the Products DataFrame
company_counts = Products['company'].value_counts()

# Print the result
print("Frequency distribution of 'company' column:")
print(company_counts)
```

- Products['company']: Selects the company column from the Products DataFrame.
- .value_counts(): Computes the count of unique values in the company column.

This will give you a series showing the counts of each unique value in the company column, providing insights into the frequency of each company in dataset.

The output will be:

```
Frequency distribution of 'company' column:
company
Unknown         5
Ntag            2
Livepath        2
Cogibox         2
Vitz            2
Browsezoom      2
Babbleopia      1
Mynte           1
Oodoo           1
Meemm           1
Wordware        1
Edgeclub        1
Jabbersphere    1
Thoughtstorm    1
```

To compute the frequency distribution of the gender column in the Customers DataFrame.

```
# Calculate the frequency distribution of the 'gender' column in
the Customers DataFrame
gender_counts = Customers['gender'].value_counts()

# Print the result
print("Frequency distribution of 'gender' column:")
print(gender_counts)
```

The output will be:

```
Frequency distribution of 'gender' column:
gender
Male      528
Female    472
Name: count, dtype: int64
```

Data Aggregations

Aggregation refers to the process of transforming a dataset by computing a single summary value (or set of values) from multiple rows or columns of data. This summary value typically represents some statistical measure or calculation applied across the dataset. Examples of aggregation operations include calculating the sum, mean (average), median, mode, minimum, maximum.

Aggregation is fundamental in data analysis for summarizing data, identifying patterns, and deriving insights from large datasets efficiently.

```
Purchase.aggregate('max')
```

The output will be:

```
id                              6000
purch_date      2019-12-06 00:00:00
customer_num                    1000
product_num                       60
amount                            20
paid                          1990.8
dtype: object
```

To use multiple aggregate functions in a single statement

```
Products['cost'].aggregate(['max','min'])
```

The output will be:

```
max    99.54
min     6.36
Name: cost, dtype: float64
```

Sorting a DataFrame

Sorting a DataFrame in pandas allows you to reorder the rows based on the values in one or more columns. This can be useful for quickly finding the highest or lowest values, organizing data in a specific order, or preparing data for further analysis.

Pandas provide sort_values() function to sort the data values of a DataFrame.

The sort_values() method in pandas can be applied to both numerical and categorical columns.

Syntax:
sort_values(by=column_name): Sorts the DataFrame by the specified column in ascending order.
sort_values(by=column_name, ascending=False): Sorts the DataFrame by the specified column in descending order.
sort_values(by=[col1, col2], ascending=[True, False]): Sorts the DataFrame by multiple columns with specified sorting orders for each column.

```
# Sort by numerical column 'amount'
sorted_purchase = Purchase.sort_values(by='amount')
print("Sorted Purchase by Amount (Ascending):\n", sorted_purchas
```

The output will be:

```
Sorted Purchase by Amount (Ascending):
         id purch_date  customer_num  product_num  amount     paid
4326  4327 2019-04-24           627           23       1    16.27
5627  5628 2019-05-31           324           58       1    73.05
5628  5629 2019-05-31           229           38       1    21.98
5629  5630 2019-05-31           139            9       1    17.11
4238  4239 2019-04-22           951           55       1     8.55
...    ...        ...           ...          ...     ...      ...
4148  4149 2019-04-21           139           14      20  1988.00
4149  4150 2019-04-21           334           44      20   613.40
2437  2438 2019-03-16           792           58      20  1241.80
2257  2258 2019-12-06           444           58      20  1461.00
708    709 2019-04-04            91           13      20   398.00

[6000 rows x 6 columns]
```

```
# Sort by categorical column 'company'
sorted_products = Products.sort_values(by='company')
print("Sorted Products by Company (Ascending):\n",
sorted_products)
```

The output will be:

```
Sorted Products by Company (Ascending):
     id                            product   cost       company
17   18                          Squid Ink  56.33         Aibox
49   50              Silicone Paper 16.5x24  63.98    Babbleopia
18   19               Loaf Pan - 2 Lb, Foil  22.83   Brainsphere
46   47                      Tobasco Sauce  48.32    Browsezoom
16   17                     Glaze - Apricot  33.78    Browsezoom
39   40                  Pepper - Cubanelle  79.31       Cogibox
29   30                        Fuji Apples  22.96       Cogibox
54   55                     Beef - Bresaola   8.55      Digitube
13   14              Cookies Oatmeal Raisin  99.40       Dynabox
1     2          Nori Sea Weed - Gold Label  85.74       Dynazzy
4     5    Wine - Shiraz Wolf Blass Premium  87.39          Eare
58   59       Nut - Almond, Blanched, Whole  74.28         Eazzy
42   43                   Pail For Lid 1537  40.69      Edgeclub
51   52          Soup - Knorr, Country Bean  98.74        Fimbee
```

```
# Sort by 'first_name' in descending order
sorted_customers = Customers.sort_values(by='first_name',
ascending=False)
sorted_customers
```

The output will be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
345	346	Zelig	Hawick	zhawick9l@nationalgeographic.com	Male	94.0	Hazelcrest	Parkway	Fresno	California	93750.0
737	738	Zara	Speed	zspeedkh@ucsd.edu	Female	88239.0	Green	Alley	Brooklyn	New York	11236.0
387	388	Zackariah	Kalkhoven	zkalkhovenar@soundcloud.com	Male	94.0	Hanson	Circle	Fort Worth	Texas	76162.0
262	263	Zack	Goult	zgoult7a@unicef.org	Male	89.0	Bluestem	Court	Miami	Florida	63121.0
909	910	Zaccaria	Finlry	zfinlryp9@umn.edu	Male	97.0	Mifflin	Trail	Worcester	Massachusetts	1654.0
...	...	...	...	...	...	...	...	...	...	...	...
935	936	Adelaida	Cullen	acullenpz@wikia.com	Female	10923.0	Logan	Plaza	Denver	Colorado	80228.0
460	461	Abran	Humbie	ahumbiecs@rediff.com	Male	3438.0	Scoville	Junction	Washington	District of Columbia	20436.0
362	363	Abel	Rodenhurst	arodenhursta2@youtu.be	Male	7578.0	Waywood	Parkway	Saint Petersburg	Florida	33710.0
833	834	Abel	Mainson	amainsonn5@ca.gov	Male	7414.0	Daystar	Alley	Boston	Utah	84135.0
270	271	Abel	Peidro	Unknown	Male	5766.0	Pearson	Park	Warren	Ohio	44485.0

1000 rows × 11 columns

```
# Sort by 'company' in ascending and 'cost' in descending order
within each company
sorted_products_multi_desc = Products.sort_values(by=['company',
'cost'], ascending=[True, False])
print("Sorted Products by Company (Asc) and Cost (Desc):\n",
sorted_products_multi_desc)
```

The output will be:

```
Sorted Products by Company (Asc) and Cost (Desc):
     id                          product   cost       company
17   18                        Squid Ink  56.33         Aibox
49   50             Silicone Paper 16.5x24  63.98    Babbleopia
18   19              Loaf Pan - 2 Lb, Foil  22.83   Brainsphere
46   47                      Tobasco Sauce  48.32    Browsezoom
16   17                    Glaze - Apricot  33.78    Browsezoom
39   40                  Pepper - Cubanelle  79.31       Cogibox
29   30                        Fuji Apples  22.96       Cogibox
54   55                    Beef - Bresaola   8.55      Digitube
13   14             Cookies Oatmeal Raisin  99.40       Dynabox
1     2          Nori Sea Weed - Gold Label  85.74       Dynazzy
4     5    Wine - Shiraz Wolf Blass Premium  87.39          Eare
58   59       Nut - Almond, Blanched, Whole  74.28         Eazzy
42   43                   Pail For Lid 1537  40.69      Edgeclub
51   52          Soup - Knorr, Country Bean  98.74        Eimbee
```

idxmax function

The idxmax function in pandas is used to return the index (or label) of the first occurrence of the maximum value. idxmax helps locate the index of the maximum value, which can be useful for retrieving related data from the same row or column.

Commonly used in data analysis for tasks like identifying the best performing entity, the highest value in time series data, or the most significant feature in a dataset.

```
index_max_Products = Products.idxmax()
index_max_Products
```

Products.idxmax() computes the index labels of the maximum values for each column in the Products DataFrame.

The output will be:

```
id          59
product      5
cost        33
company     22
dtype: int64
```

```
# Finding the index of the maximum value in the 'paid' column
index_max_paid = Purchase['paid'].idxmax()

print('Index of the maximum paid amount:', index_max_paid)
```

idxmax() is used on the 'paid' column to find the index of the row with the highest value in that column.

This code will output the index of the row with the highest paid amount and display the entire row.

The output will be:

```
Index of the maximum paid amount: 211
```

GROUP BY Functions

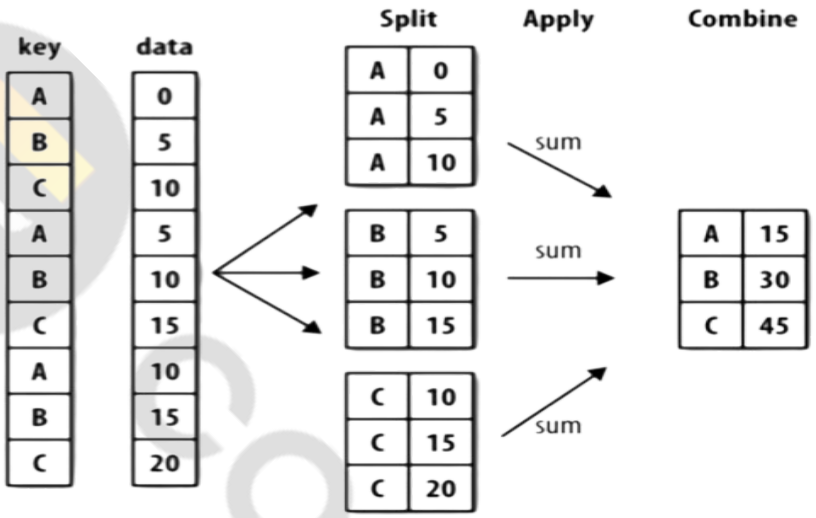
Grouping in pandas allows us to aggregate data based on one or more columns and then apply functions (such as sum, mean, count) to each group.

Grouping involves the following steps:

- Splitting: Data is divided into groups based on some criteria.
- Applying: A function is applied to each group independently.
- Combining: Results are combined into a DataFrame, Series, or aggregated result.

To understand this better, let us consider the data shown in the diagram given below. Here, we have a two column DataFrame (key, data). We need to find the sum of the data column for

a particular key, i.e. sum of all the data elements with key A, B and C, respectively. To do so, we first split the entire DataFrame into groups by key column. Then, we apply the sum function on the respective groups. Finally, we combine the results to form a new DataFrame that contains the desired result.



The following statements show how to apply GROUP BY() function on our DataFrame

```
# Group by 'company' column
grouped_by_company = Products.groupby('company')

# Calculate average cost of products per company
average_cost = grouped_by_company['cost'].mean()

# Calculate total number of products per company
total_products = grouped_by_company['product'].count()

# Display the results
print("Average Cost of Products per Company:\n", average_cost)
print("\nTotal Number of Products per Company:\n", total_products)
```

The output will be:

```
Average Cost of Products per Company:
 company
Aibox           56.330
Babbleopia      63.980
Brainsphere     22.830
Browsezoom      41.050
Cogibox         51.135
Digitube         8.550
Dynabox         99.400
Dynazzy         85.740
Eare            87.390
Eazzy           74.280
Edgeclub        40.690
```

```
Total Number of Products per Company
 company
Aibox           1
Babbleopia      1
Brainsphere     1
Browsezoom      2
Cogibox         2
Digitube        1
Dynabox         1
Dynazzy         1
Eare            1
Eazzy           1
```

- Splitting: The DataFrame Products is split into groups based on unique values in the 'company' column.
- Applying: Aggregate functions like .mean() and .count() are applied to the 'cost' and 'product' columns within each group, respectively.
- Combining: Results are combined into Series where each index corresponds to a unique company.

```
# Group by 'purch_date' and calculate the total 'paid' amount per
date
total_paid_per_date = Purchase.groupby('purch_date')['paid'].sum()

# Group by 'product_num' and calculate the average 'amount'
purchased per product
avg_amount_per_product =
Purchase.groupby('product_num')['amount'].mean()

# Print the results
print("Total paid amount per date:")
print(total_paid_per_date)
print("\nAverage amount purchased per product:")
print(avg_amount_per_product)
```

The output will be:

```
Total paid amount per date:
purch_date
2019-01-03    20468.33
2019-01-04    48529.13
2019-01-05    38643.67
2019-01-06    32345.29
2019-02-03    28183.50
                ...
2019-11-06    24778.05
2019-12-03    27691.62
2019-12-04    37100.59
2019-12-05    20845.55
2019-12-06    26410.08
Name: paid, Length: 108, dtype: float64

Average amount purchased per product:
product_num
1      9.958763
2     10.861386
3      9.754902
4     10.126214
5     11.096774
6     11.520408
7     10.525773
8     10.609091
9     10.467391
10    10.391304
```

- We grouped by purch_date to calculate the total paid amount per date using sum().
- We grouped by product_num to calculate the average amount purchased per product using mean().

Altering the Index

To alter or set a specific column as the index in a DataFrame in pandas, we can use the set_index() method. This allows you to rearrange your DataFrame so that the specified column becomes the index.

```
# Set 'id' column as index
Customers.set_index('id', inplace=True)
print("\nDataFrame with 'id' as index:")
Customers
```

The output will be:

```
DataFrame with 'id' as index:
```

	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
id										
1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0
2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	1.0	Trailsway	Road	El Paso	Texas	78220.0
3	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0
4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0
5	Mikaela	Althorpe	malthorpe4@51.la	Male	74.0	2nd	Drive	Baltimore	Maryland	83732.0
...	...	...	...	...	...	...	...	...	...	...
996	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0
997	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0
998	Tobit	Birt	Unknown	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0
999	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	2.0	Kenwood	Drive	Savannah	Georgia	31405.0
1000	Elmore	Malpas	Unknown	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0

1000 rows × 10 columns

To reset the index of a DataFrame in pandas, we can use the reset_index() method. This method is useful when you want to move the current index levels back into columns and reset the index to the default integer index (0, 1, 2, ...).

```
# Reset index
Customers.reset_index(inplace=True)
print("\nDataFrame after resetting index:")
Customers
```

The output will be:

```
DataFrame after resetting index:
```

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
0	1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0
1	2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	1.0	Trailsway	Road	El Paso	Texas	78220.0
2	3	Bambi	Westrip	bwestrip2@symantec.com	Female	4057.0	Arkansas	Circle	San Antonio	Texas	78220.0
3	4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0
4	5	Mikaela	Althorpe	malthorpe4@51.la	Male	74.0	2nd	Drive	Baltimore	Maryland	83732.0
...	...	...	...	...	...	...	...	...	...	...	...
995	996	Merrili	Alman	malmanrn@cornell.edu	Female	0.0	Thompson	Way	Reading	Pennsylvania	19610.0
996	997	Winonah	Heckle	whecklero@fc2.com	Female	701.0	Rowland	Hill	Indianapolis	Indiana	46278.0
997	998	Tobit	Birt	Unknown	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0
998	999	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	2.0	Kenwood	Drive	Savannah	Georgia	31405.0
999	1000	Elmore	Malpas	Unknown	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0

1000 rows × 11 columns

<For instructor, the below concepts are very important so please teach the concepts and ask students to practice it with examples from the dataset provided>

Adding important new concepts

- **pivot**: Ideal for reshaping and aggregating data, offering more complex and detailed summaries with the ability to handle multi-level indexing.
- **crosstab**: Best for frequency distributions and simple cross-tabulations of categorical data. It is straightforward for summarizing categorical relationships.
- **melt**: Perfect for transforming data from wide to long format without aggregation, making it useful for data normalization and preparation for further analysis.

1. **PIVOT TABLES**
   A pivot table in pandas is a powerful tool that allows you to summarize and aggregate data in a flexible and customizable way. It is particularly useful for exploring large datasets and identifying patterns or trends.

   Key Concepts

- **Data Aggregation**: Pivot tables allow you to aggregate data using functions like sum, mean, count, min, max, etc.
- **Grouping**: Data can be grouped by one or more columns, enabling multi-level summaries.
- **Pivoting**: Columns can be pivoted to become rows, providing a different perspective of the data.
- **Multi-indexing**: Pivot tables can create multi-level indices for rows and columns, offering more detailed and hierarchical views.

  Example -

```
import pandas as pd

# Sample DataFrame
data = {
    'Region': ['East', 'West', 'East', 'West', 'East', 'West'],
    'Product': ['A', 'A', 'B', 'B', 'C', 'C'],
    'Sales': [100, 150, 200, 250, 300, 350]
}
df = pd.DataFrame(data)

# Creating a pivot table
pivot_table = df.pivot_table(values='Sales', index='Region', columns='Product',
aggfunc='sum', fill_value=0)
print(pivot_table)
```

```
Output of the above -

Product    A    B    C
Region
East     100  200  300
West     150  250  350
```

Custom aggregation function can be created and used within pivot tables

```
def custom_aggfunc(x):
    return x.max() - x.min()

pivot_table = df.pivot_table(values='Sales', index='Region',
columns='Product', aggfunc=custom_aggfunc)
```

Parameters

- index: array-like, Series, or list of arrays/Series
  - Values to group by in the rows.
  - Multiple index values can also be provided for multi-level indexing index=['Region', 'Product']
- columns: array-like, Series, or list of arrays/Series
  - Values to group by in the columns.
- values: array-like, optional
  - Array of values to aggregate.
- aggfunc: function, optional, default numpy.mean
  - Multiple aggregation function can be used; aggfunc=[sum, 'mean'].
- margins: bool, default False
  - Add all row/columns (e.g., for subtotal/grand totals).
- normalize: bool, {'all', 'index', 'columns'}, or {0,1}, default False
  - Normalize by dividing all values by the sum of values.

2. CROSS TAB

Crosstab (cross-tabulation) in pandas is used to compute a simple cross-tabulation of two (or more) factors. It is similar to a pivot table but focuses on count frequencies.

Use Cases

- Counting occurrences of categorical data.
- Analyzing contingency tables in survey data.

Code -

```
import pandas as pd
# Sample DataFrame
data = {
    'Region': ['East', 'West', 'East', 'West', 'East', 'West'],
    'Product': ['A', 'A', 'B', 'B', 'C', 'C'],
    'Sales': [100, 150, 200, 250, 300, 350]
}
df = pd.DataFrame(data)
# Creating a crosstab
crosstab = pd.crosstab(df['Region'], df['Product'])
print(crosstab)
```

Output

```
Product  A  B  C
Region
East     1  1  1
West     1  1  1
```

Different parameters of cross-tab:

- index: array-like, Series, or list of arrays/Series Values to group by in the rows.
- columns: array-like, Series, or list of arrays/Series; Values to group by in the columns.
- values: array-like, optional; Array of values to aggregate.
- aggfunc: function, optional, default numpy.mean; Aggregation function to use.
- margins: bool, default False; Add all row/columns (e.g., for subtotal/grand totals).
- normalize: bool, {'all', 'index', 'columns'}, or {0,1}, default False; Normalize by dividing all values by the sum of values.

3. MELTING

Melting in pandas is used to transform a DataFrame from a wide format to a long format. This is useful for data normalization and making data suitable for certain types of analysis.

```
import pandas as pd
# Sample DataFrame
data = {
    'Region': ['East', 'West'],
    'Product_A': [100, 150],
    'Product_B': [200, 250],
    'Product_C': [300, 350]
}
df = pd.DataFrame(data)
# Melting the DataFrame
melted_df = pd.melt(df, id_vars=['Region'], value_vars=['Product_A', 'Product_B',
'Product_C'], var_name='Product', value_name='Sales')
print(melted_df)
```

Output

```
  Region    Product  Sales
0   East  Product_A    100
1   West  Product_A    150
2   East  Product_B    200
3   West  Product_B    250
4   East  Product_C    300
5   West  Product_C    350
```

Parameters

- frame: DataFrame
    - The DataFrame to unpivot.
- id_vars: tuple, list, or ndarray
    - Column(s) to use as identifier variables.
- value_vars: tuple, list, or ndarray, optional
    - Column(s) to unpivot.
- var_name: scalar, default None
    - Name to use for the ‘variable’ column.
- value_name: scalar, default 'value'
    - Name to use for the ‘value’ column.

Comparing the three methods below -

Feature	pivot	crosstab	melt
Purpose	Reshape data by pivoting columns into rows and aggregating data.	Create a contingency table to show the frequency distribution of categorical data.	Transform DataFrame from wide format to long format.
Use Cases	Summarizing sales data, financial reports, survey analysis.	Analyzing survey data, market research, social sciences.	Preparing data for statistical analysis, normalizing data for ML.
Input Data	DataFrame with unique values in the specified index/column.	Series or array-like (categorical variables).	DataFrame in wide format.
Aggregation	Yes, using the aggfunc parameter (e.g., sum, mean, count).	Yes, using the aggfunc parameter (e.g., sum, mean, count).	No aggregation, simply reshaping data.
Parameters	index, columns, values, aggfunc, fill_value, margins, dropna.	index, columns, values, aggfunc, margins, normalize, dropna.	id_vars, value_vars, var_name, value_name.
Normalization	No direct normalization parameter.	Yes, using the normalize parameter to divide values by the sum of values.	No normalization, just reshaping.
Handling Missing Data	Handles missing data with fill_value parameter.	Handles missing data with dropna parameter.	Does not handle missing data directly, reshapes including all data.
Output Format	DataFrame (pivoted and aggregated table).	DataFrame (contingency table).	DataFrame (long format).
Complexity	More complex, suitable for detailed summaries and aggregations.	Relatively simple for frequency distribution.	Simple transformation, no aggregation.
Multi-index	Can create multi-index for rows and columns.	Can create multi-index for rows and columns.	Not applicable, single-level index.
Example	df.pivot(index='Region', columns='Product', values='Sales')	pd.crosstab(df['Region'], df['Product'])	pd.melt(df, id_vars=['Region'], value_vars=['Product_A', 'Product_B', 'Product_C'])

Common Use with Aggregation	Yes, commonly used with sum, mean, etc.	Yes, especially for count and mean.	No, typically used for reshaping without aggregation.

Understand how to export a DataFrame to a CSV or Excel file

Exporting to a CSV file

To export a DataFrame to a CSV file, use the to_csv() method. This method allows you to specify the file path and various options for formatting the output.

Syntax

```
# Export DataFrame to a CSV file
df.to_csv('output.csv', index=False)
```

- to_csv('output.csv', index=False): This exports the DataFrame df to a file named output.csv. The index=False parameter ensures that the index is not included in the CSV file.

Exporting to an Excel file

To export a DataFrame to an Excel file, use the to_excel() method.

Syntax:

```
# Export DataFrame to an Excel file
df.to_excel('output.xlsx', index=False)
```

- to_excel('output.xlsx', index=False): This exports the DataFrame df to a file named output.xlsx. The index=False parameter ensures that the index is not included in the Excel file.